

Git e GitHub na Prática

Controle de versão, armazenamento de código e colaboração.

Turma Programação Full-Stack

Treina Recife | Docente: Tereza Oliveira

Cronograma da Aula

4 horas de imersão prática: Fundamentos, comandos essenciais, colaboração e desafio final.

Fundamentos: Git vs GitHub



O que é o Git?

O Git é um sistema de controle de versão local. Funciona como um histórico organizado do projeto, registrando cada momento importante do desenvolvimento e evitando a perda de código.



O que é o GitHub?

Uma plataforma em nuvem (internet) que armazena seus repositórios Git. Permite colaboração entre desenvolvedores, revisão de código e funciona como seu portfólio profissional.



O Fluxo Básico

O trabalho contínuo no Git segue uma sequência lógica essencial e ininterrupta: você vai **Modificar** o arquivo, **Preparar** a alteração e, por fim, **Registrar** essa versão no histórico.

Prática 1: Preparação do Ambiente

O Passo a Passo

1. Crie ou acesse sua conta gratuita no GitHub.
2. Abra o Visual Studio Code (VS Code).
3. Abra o Terminal integrado no VS Code.
4. Verifique a instalação e configure seu perfil usando os comandos no terminal ao lado.

Importante: O e-mail configurado no Git deve ser o mesmo utilizado na sua conta do GitHub para vincular corretamente os commits.

Terminal (Comandos)

Verifique a instalação do Git:

```
git --version
```

Configure suas credenciais globais:

```
git config --global user.name "Seu Nome"  
git config --global user.email  
"email@exemplo.com"
```

Para conferir se a configuração deu certo:

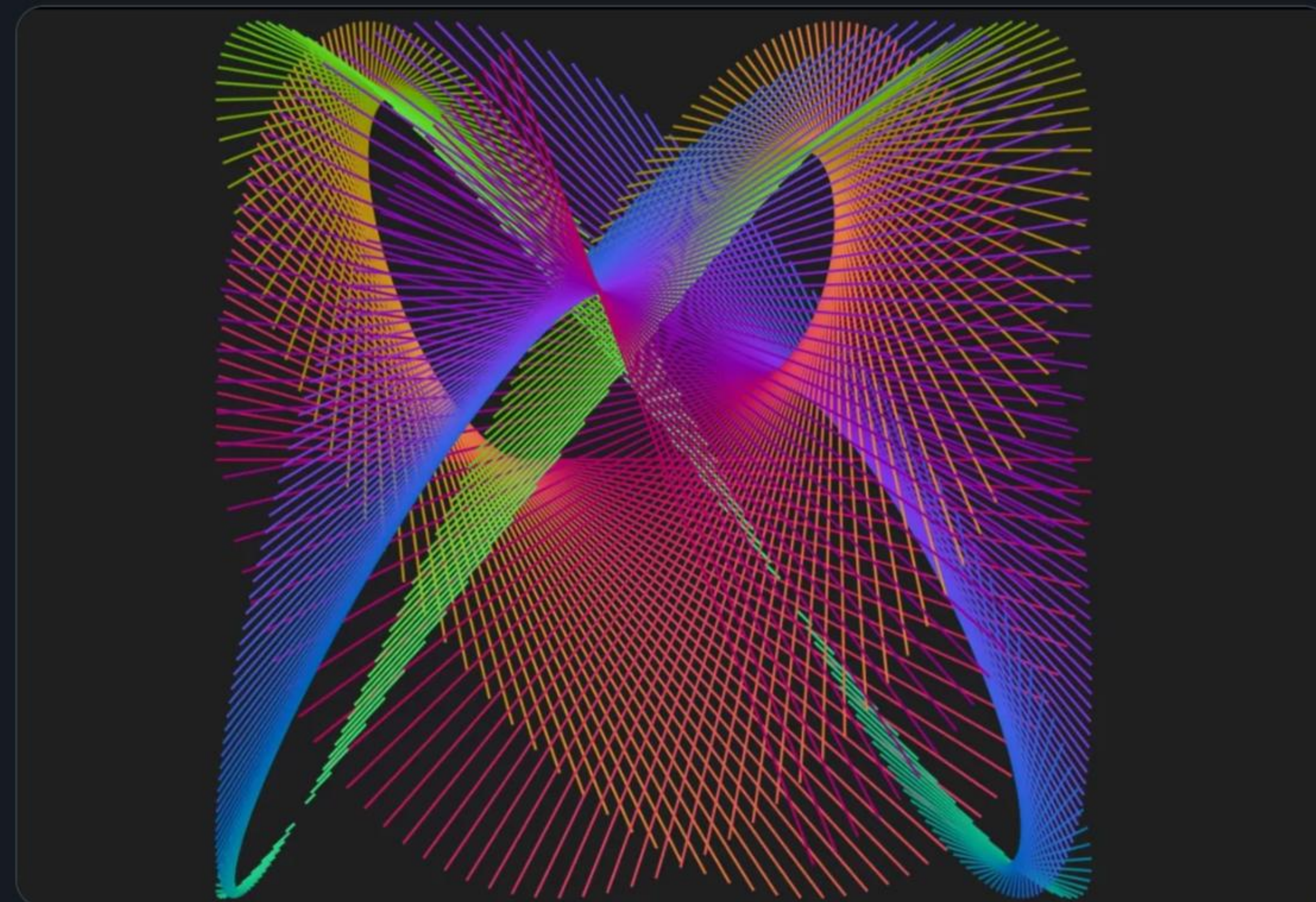
```
git config --global --list
```


Cheatsheet: Comandos Essenciais

Comando	Função (O que faz)
<code>git init</code>	Inicia o controle de versão na pasta atual (cria repositório local).
<code>git status</code>	Exibe o estado dos arquivos (modificados, adicionados, não mapeados).
<code>git add .</code>	Prepara as alterações para o próximo commit.
<code>git commit -m "Mensagem"</code>	Registra uma versão definitiva no histórico do projeto.
<code>git push</code>	Envia as alterações locais (commits) para o GitHub (repositório remoto).
<code>git pull</code>	Baixa e integra as alterações do GitHub para a sua máquina local.
<code>git clone [URL]</code>	Cria uma cópia exata de um repositório remoto na sua máquina.

Prática 2: Primeiro Repositório

- > Crie uma pasta local: `mkdir meu-primeiro-projeto`.
- > Inicialize o controle de versão: `git init`.
- > Crie uma pequena página de apresentação pessoal com `index.html` e `style.css`.
- > Adicione e registre a criação: `git add .` seguido de `git commit -m "Adiciona estrutura inicial"`.
- > No GitHub, clique em "New repository" (Repositório Público).
- > Conecte sua pasta local ao GitHub e envie o código: `git remote add origin URL` e `git push -u origin main`.



Trabalho Colaborativo: Branches

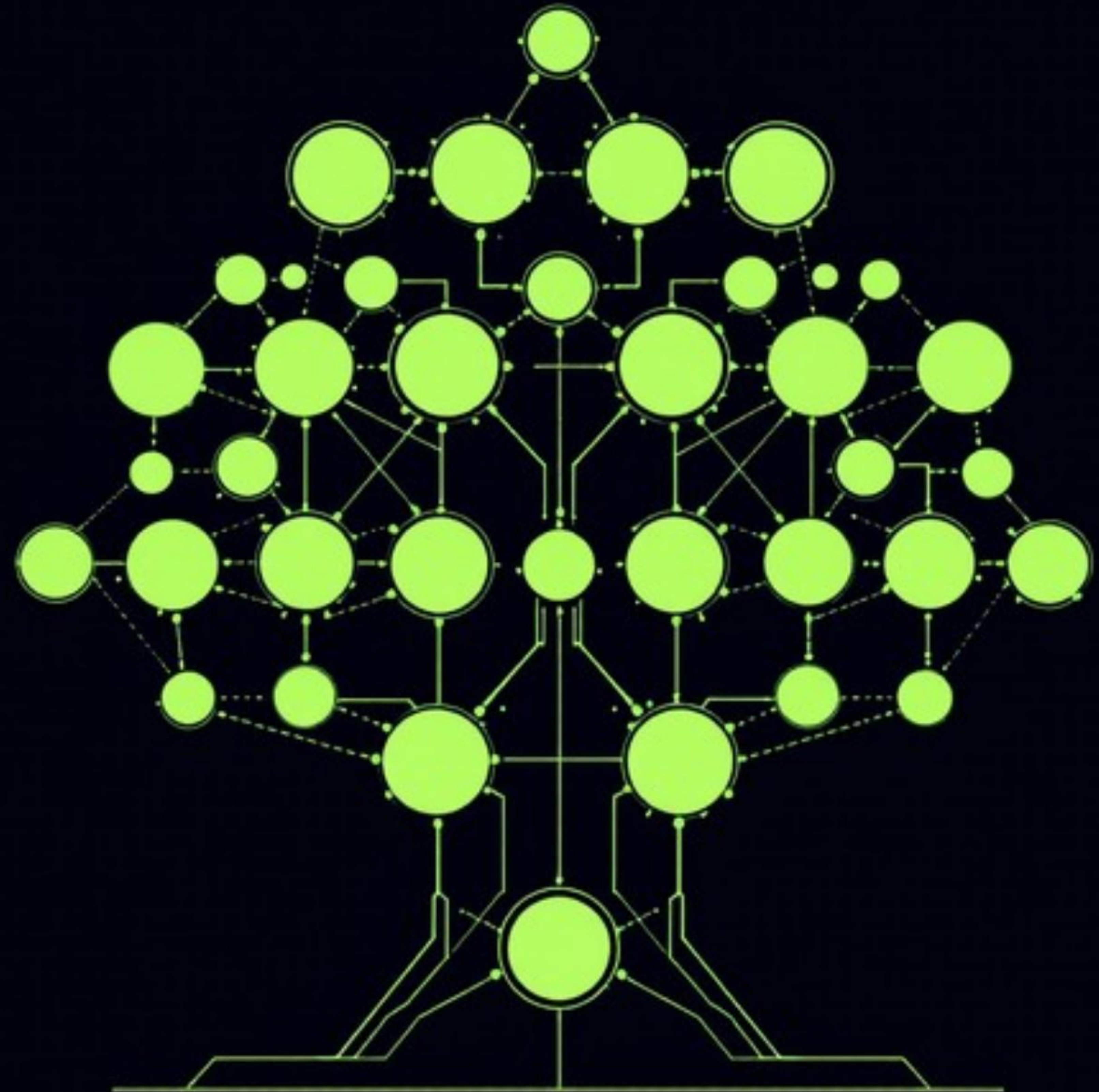
Por que não desenvolver tudo na `main`? As ramificações (branches) permitem testar e construir funcionalidades de forma segura e paralela, sem quebrar o código principal.

Padrões de Nomenclatura sugeridos:

- > feature/adiciona-contato
- > feature/cria-menu
- > fix/corrige-link
- > docs/atualiza-readme

Criar e mudar de branch simultaneamente:

```
git switch -c feature/nova-secao
```



Integração: Pull Request (PR)



Solicitar Revisão

Uma PR é uma solicitação para integrar as alterações da sua branch ao projeto principal. Ela permite revisar o código, conversar sobre as alterações e pedir ajustes à dupla.



Resolver Conflitos

Conflitos ocorrem quando duas branches alteram a mesma linha de um arquivo. Você deverá ler as duas versões (HEAD vs branch), decidir qual manter, e registrar a correção.



Realizar o Merge

O Merge é o processo de consolidar (juntar) as ramificações. Após a revisão da PR e a resolução de todos os conflitos, o código da branch é oficialmente integrado à principal.

Boas Práticas: Commits e README



Mensagens de Commit Claras e Diretas

Um bom commit deve ser curto, específico e representar uma alteração coerente. Evite mensagens como `"ajustes"`, `"teste"`, ou `"agora vai"`. Prefira mensagens utilitárias como: `"Adiciona formulário de contato"` ou `"Corrige alinhamento do menu"`.



O README é a Porta de Entrada

O arquivo `README.md` é a primeira coisa que visitantes (e recrutadores) leem ao abrir seu repositório. Uma boa documentação valoriza seu código e demonstra capricho.



Estrutura Mínima do README

Certifique-se de incluir um Título Claro, Tecnologias Utilizadas (HTML, CSS, etc), Funcionalidades do projeto, Como executar o código e a Autoria. Utilize Markdown para a formatação.

Prática 3: O Desafio Final

O que entregar?

Cada participante deverá construir e hospedar um repositório contendo o fluxo completo:

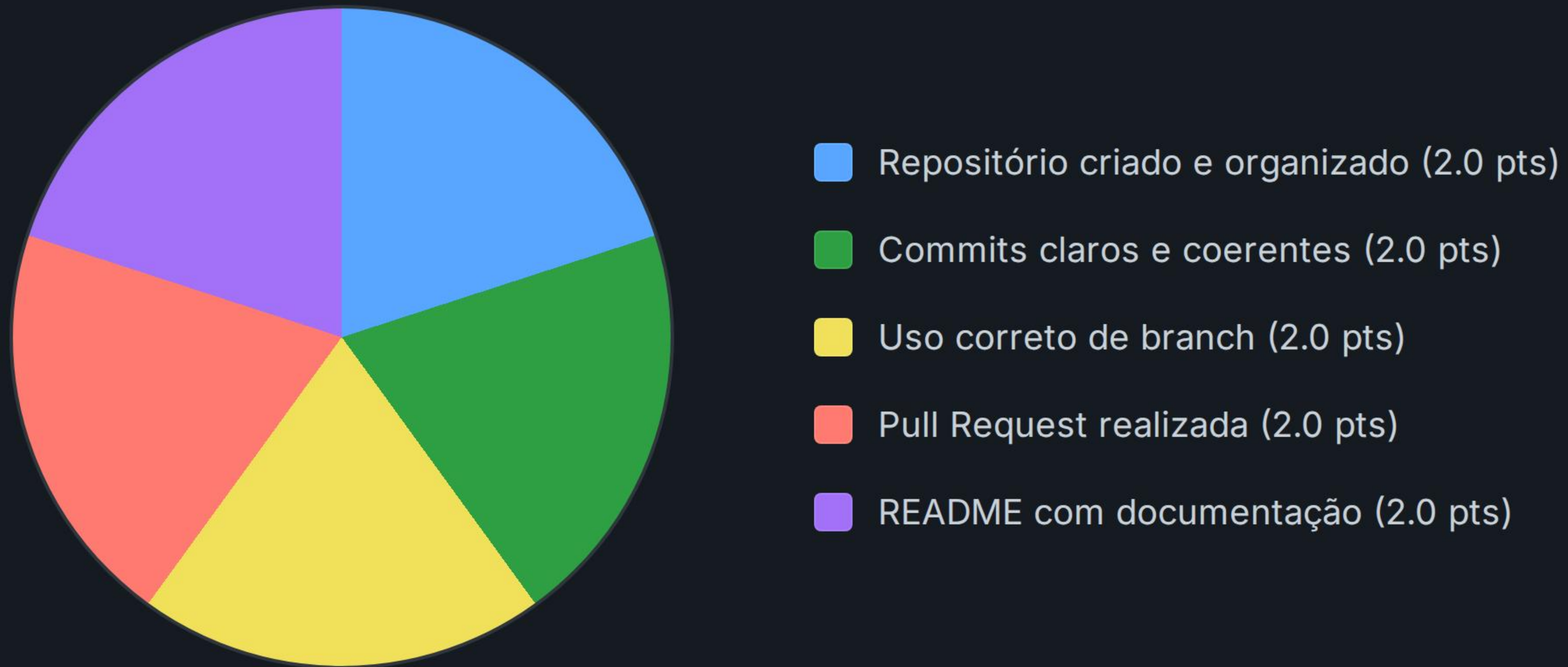
- > Página pessoal em HTML e CSS.
- > No mínimo **três commits** registrados.
- > Criação de pelo menos **uma branch** secundária.
- > **Uma Pull Request** aberta e concluída.

Requisitos de Qualidade

A avaliação levará em consideração a organização geral do seu projeto e perfil:

- > Arquivo `README.md` padronizado e documentado.
- > Histórico de alterações e commits semânticos.
- > Link do repositório deve estar como **Público** no GitHub para a avaliação da docente.

Critérios de Avaliação (10.0 Pontos)



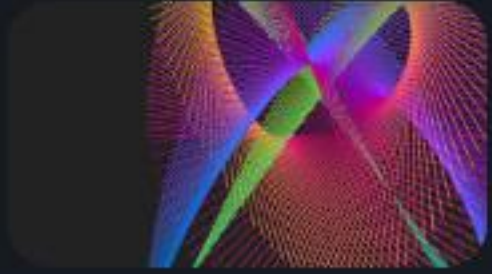
A avaliação acontecerá durante toda a aula, observando sua participação prática, execução de comandos e colaboração com a dupla.

Hora de Praticar!

Dúvidas? Mãos à obra.

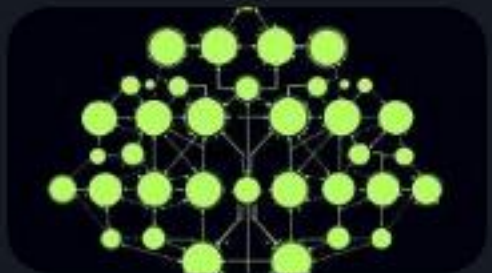
Treina Recife - Programação Full-Stack
Docente: Tereza Oliveira

Image Sources



<https://codetap.org/storage/projects/simple-line-art-threejs.webp>

Source: codetap.org



https://images.stockcake.com/public/f/5/4/f54572a0-134a-4a9d-953e-c1594ca9140b_large/glowing-decision-tree-stockcake.jpg

Source: stockcake.com